

INFO145 - Diseño y Análisis de Algoritmos

12 - Algoritmos Greedy II

Héctor Ferrada
académico

Instituto de Informática
Universidad Austral de Chile

The Shortest-Path Problem

Dados:

1. Un **grafo dirigido** $G = (V, E)$; donde V es el conjunto de vértices y E es el conjunto de arcos (u, v) , con $u, v \in V$.
2. Una función de peso, o costo, asociada a los arcos, denotada por:
 $\omega : E \rightarrow \mathbb{R}^+$, donde $\omega(u, v)$ es el costo de ir de u a v .

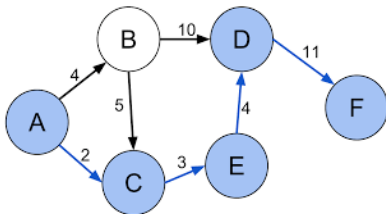
Definimos un camino o ruta p de v_0 a v_k como $p = \langle v_0, v_1, \dots, v_k \rangle$, y al costo de este camino p como:

$$\omega(p) = \sum_{i=1}^k \omega(v_{i-1}, v_i)$$

- El problema de encontrar el camino que minimiza la suma de los costos desde u hasta v , se llama *Shortest-Path Problem* y viene dado por:

$$\delta(u, v) = \begin{cases} \min\{\omega(p) : p \text{ es un camino de } u \text{ a } v\} \\ \infty \text{ Si } \nexists \text{ camino de } u \text{ a } v \end{cases}$$

Ejemplo Shortest-Path Problem



En la imagen, se destaca en azul la ruta más corta desde *A* hasta *F*.

Algoritmo de Dijkstra

Encuentra los caminos de costo mínimo desde un vértice s a todos los restantes, en un grafo dirigido y ponderado G .

Lema. Subcaminos de caminos de costo mínimo son también caminos de costo mínimo. Esto viene derivado del teorema de Kruskal

Demostración. Sea p un camino de costo mínimo desde v_0 a v_k , descompuesto en:

$$p_{0i} : v_0 \longrightarrow v_i$$

$$p_{ij} : v_i \longrightarrow v_j$$

$$p_{jk} : v_j \longrightarrow v_k$$

$$\text{Entonces: } \omega(p) = \omega(p_{0i}) + \omega(p_{ij}) + \omega(p_{ik})$$

Suponga ahora que existe p'_{ij} alternativo a p_{ij} con peso $\omega(p'_{ij}) < \omega(p_{ij})$

$$\Rightarrow \omega(p_{0i}) + \omega(p'_{ij}) + \omega(p_{ik}) < \omega(p) \rightarrow \leftarrow$$

Contradicción ya que p es mínimo. Q.E.D.

Algoritmo de Dijkstra

Input: Grafo dirigido $G = (V, Ad)$, donde Ad es la lista de adyacencia, el nodo inicial s y la función de costo $\omega(\cdot) : E \rightarrow \mathbb{R}^+$

Output: Grafo S con los ancestros y distancias para todos los caminos de costo mínimo

```
DIJKSTRA( $G = (V, Ad)$ ,  $s$ ,  $\omega$ ) {  
    for each  $u \in V$  do {  
         $u.dist = \infty$            //distancia  
         $u.anc = \text{NULL}$          //ancestro  
    }  
     $s.dist = 0$   
     $Q \leftarrow V$                 //Q siempre está ordenado por distancias  
     $S = \emptyset$   
    while ( $Q \neq \emptyset$ ) do {  
         $u = \text{EXTRACT-MIN}(Q)$  //note que al inicio se extrae s  
         $S = S \cup \{u\}$   
        for each  $v \in G.Ad[u]$  do {  
            if ( $v.dist > u.dist + \omega(u, v)$ ) then {  
                 $v.dist = u.dist + \omega(u, v)$   
                 $v.anc = u$   
                ACTUALIZAR( $Q, v$ )    //dejar v bien posicionado en Q  
            }  
        }  
    }  
}
```

Algoritmo de Dijkstra

Para el análisis, nuevamente utilizaremos una cola de prioridad, cuyas operaciones toman un tiempo de:

$Q = \text{MAKE-MIN}(V) \rightarrow O(|V|)$

$\text{EXTRACT-MIN}(Q)$ y $\text{ACTUALIZAR}(Q, v) \rightarrow O(\log |V|)$

Sean $n = |V|$ y $m = |E|$, entonces:

1. El primer for each toma $O(|V|) = O(n)$
2. El while itera n veces, pues los nodos se extraen una sola vez de Q .
3. Dentro del ciclo while tenemos:

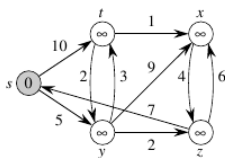
3.1 $\text{EXTRACT-MIN}(Q)$ en $O(\log n)$

3.2 El segundo for each, en donde vemos que la cantidad de arcos e , para toda la ejecución del algoritmo, es m .

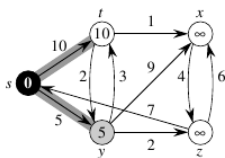
Además, dentro de este ciclo la operación $\text{ACTUALIZAR}(Q, v)$ toma $O(\log n)$, esto es añadiendo otro campo en el nodo para almacenar que posición tiene este en Q . Así, lo ubicamos en $O(1)$ en el heap y lo dejamos bien posicionado en $O(\log n)$, actualizando también la posición de los nodos que se muevan en esta operación.

En total: $O(n + n \log n + m \log n) = O((n + m) \log n)$

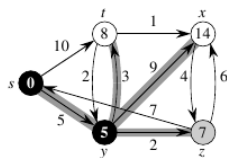
Algoritmo de Dijkstra - Ejemplo



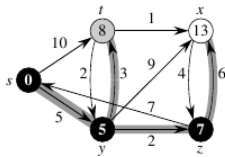
(a)



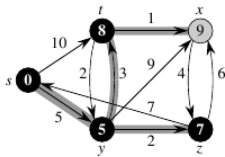
(b)



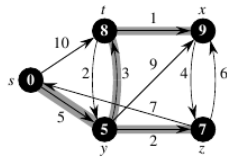
(c)



(d)



(e)



(f)

En la imagen, los nodos en negro son los que se unen a S . Note que la estrategia greedy es que Dijkstra siempre elige al nodo más *liviano* o *cercano* en $V - S$ para añadir a S .

All Pairs Shortest-Path Problem

Dados:

1. Un **grafo dirigido** $G = (V, E)$; donde V es el conjunto de vértices y E es el conjunto de arcos (u, v) , con $u, v \in V$.
2. Una función de peso, o costo, asociada a los arcos, denotada por:
 $\omega : E \rightarrow \mathbb{R}$; tal que no existen ciclos de costo negativo en G .

Se desea encontrar el camino de costo mínimo **para todo par de vértices en V** .

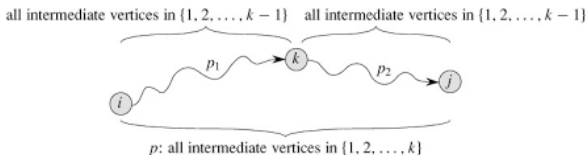
Denotamos un ciclo de costo negativo como: $\delta(u, v) = -\infty$

Note que, para resolver este problema, no es posible utilizar Dijkstra ya que ahora debemos considerar costos negativos.

Algoritmo de Floyd-Warshall

Resuelve All Pairs Shortest-Path Problem de la siguiente manera:

- La idea es considerar que en el camino de costo mínimo entre u y v pueden haber nodos (vértices) intermedios.
- **Estrategia Greedy:** el primer paso no permite nodos intermedios; es decir, el mejor camino entre u y v será la arista $e = (u, v)$ si existe en G . Luego, gradualmente permitimos la utilización de otros nodos intermedios, incorporando un nodo en cada paso y actualizamos las distancias.



Entonces $V = \{v_1, v_2, \dots, v_n\}$ y denotamos d_{ij}^k (usaremos la matriz $dist(i, j, k)$) como la distancia más corta desde v_i a v_j considerando solamente utilizar vértices en $\{v_1, v_2, \dots, v_k\}$ como nodos intermedios. Así:

$$dist(i, j, k) = d_{ij}^k = \begin{cases} \omega(i, j) & \text{si } k = 0 \\ \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) & \text{si } k > 0 \end{cases}$$

Algoritmo de Floyd-Warshall

Input: Grafo dirigido $G = (V, E)$, y la función de costo $\omega(\cdot) : E \rightarrow \mathbb{R}$

Output: Matriz $dist[1..n][1..n][1..n]$ de distancias mínimas para todo par de vértices

FLOYD-WARSHALL($G = (V, e), \omega$) {

$n = |V|$

 for $i = 1$ to n do

 for $j = 1$ to n do

$dist(i, j, 0) = \infty$ *// inicialización de la matriz*

 for each $e = (u, v) \in G.E$ do

$dist(u, v, 0) = \omega(u, v)$ *// sin considerar nodos intermedios*

 for $k = 1$ to n do

 for $i = 1$ to n do

 for $j = 1$ to n do

// ahora consideramos incluir a v_k como nodo intermedio

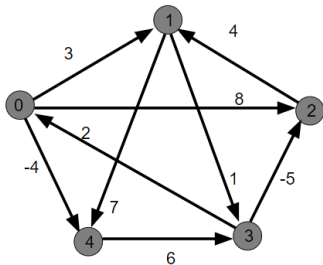
$dist(i, j, k) = \min\{dist(i, j, k-1), dist(i, k, k-1) + dist(k, j, k-1)\}$

 return $dist$

}

Claramente su tiempo de ejecución es $O(n^3)$

Algoritmo de Floyd-Warshall - Ejemplo



	0	1	2	3	4
0	0	3	8	∞	-4
1	∞	0	∞	1	7
2	∞	4	0	∞	∞
3	2	∞	-5	0	∞
4	∞	∞	∞	6	0

0	1	2	3	4
0	3	8	∞	-4
1	0	∞	1	7
2	4	0	∞	∞
3	5	-5	0	-2
4	∞	∞	6	0

0	1	2	3	4
0	3	8	4	-4
1	0	∞	1	7
2	4	0	5	11
3	5	-5	0	-2
4	∞	∞	6	0

0	1	2	3	4
0	3	8	4	-4
1	0	∞	1	7
2	4	0	5	11
3	-1	-5	0	-2
4	∞	∞	6	0

0	1	2	3	4
3	0	-1	4	-4
7	4	0	5	3
2	-1	-5	0	-2
8	5	1	6	0

0	1	-3	2	-4
3	0	-4	1	-1
7	4	0	5	3
2	-1	-5	0	-2
8	5	1	6	0

Problema de Selección de Actividades

Se desea seleccionar el mayor número de actividades que puede realizar una sola persona, asumiendo que una persona solo puede trabajar en una sola actividad a la vez.

Para esto:

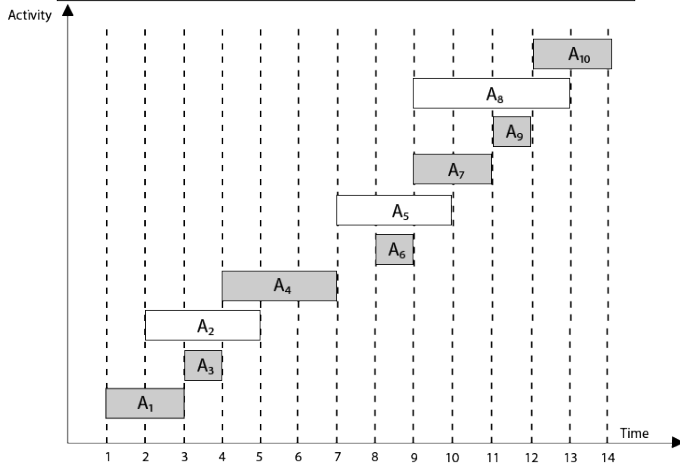
- Se dispone de n actividades con sus horas de inicio y de término.
- La persona puede asistir a la tarea que desee.
- Pueden haber actividades que en sus tiempos hay solapamiento, por tanto solo podrá asistir a una de ellas.

¿Qué es lo que deseamos priorizar?

- P1: ¿Asistir al mayor número de actividades posible?.
- P2: ¿Minimizar el tiempo que la persona esta inactiva?.

Ejemplo

Activity	A ₁	A ₃	A ₂	A ₄	A ₆	A ₅	A ₇	A ₉	A ₈	A ₁₀
Start	1	3	2	4	8	7	9	11	9	12
Finish	3	4	5	7	9	10	11	12	13	14



Estrategias Greedy Para la P1

Tomar una tarea en cada paso de acuerdo a uno de los siguientes criterios:

1. La primera actividad disponible según el orden creciente de las horas de comienzo.

Estrategias Greedy Para la P1

Tomar una tarea en cada paso de acuerdo a uno de los siguientes criterios:

1. La primera actividad disponible según el orden creciente de las horas de comienzo.



Estrategias Greedy Para la P1

Tomar una tarea en cada paso de acuerdo a uno de los siguientes criterios:

1. La primera actividad disponible según el orden creciente de las horas de comienzo.



2. La primera actividad disponible según el orden creciente de la duración de cada actividad $\Rightarrow t_{fin}^i - t_{ini}^i, \forall i$ con $1 \leq i \leq n$.

Estrategias Greedy Para la P1

Tomar una tarea en cada paso de acuerdo a uno de los siguientes criterios:

1. La primera actividad disponible según el orden creciente de las horas de comienzo.



2. La primera actividad disponible según el orden creciente de la duración de cada actividad $\Rightarrow t_{fin}^i - t_{ini}^i, \forall i$ con $1 \leq i \leq n$.



Estrategias Greedy Para la P1

Tomar una tarea en cada paso de acuerdo a uno de los siguientes criterios:

1. La primera actividad disponible según el orden creciente de las horas de comienzo.



2. La primera actividad disponible según el orden creciente de la duración de cada actividad $\Rightarrow t_{fin}^i - t_{ini}^i, \forall i$ con $1 \leq i \leq n$.



3. La primera actividad disponible según el orden creciente del número de conflictos con otras actividades.

Estrategias Greedy Para la P1

Tomar una tarea en cada paso de acuerdo a uno de los siguientes criterios:

1. La primera actividad disponible según el orden creciente de las horas de comienzo.



2. La primera actividad disponible según el orden creciente de la duración de cada actividad $\Rightarrow t_{fin}^i - t_{ini}^i, \forall i$ con $1 \leq i \leq n$.



3. La primera actividad disponible según el orden creciente del número de conflictos con otras actividades.



Estrategia Greedy Óptima.

- Tomar la primera actividad disponible según el orden creciente de sus horas de término.

Estrategia Greedy Óptima.

- Tomar la primera actividad disponible según el orden creciente de sus horas de término.

Ejemplo 1:



Ejemplo 2:



Ejemplo 3:



Estrategia Greedy Óptima.

- Tomar la primera actividad disponible según el orden creciente de sus horas de término.

Ejemplo 1:



Ejemplo 2:



Ejemplo 3:



1- Escriba el pseudocódigo de este algoritmo greedy y determine su tiempo de complejidad.

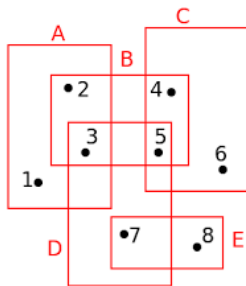
2- ¿Cómo se adapta para incluir la P2 en segundo lugar?

Minimum Set Cover Problem (MSCP)

Dado un universo $\mathcal{X}$ y una familia de conjuntos $\mathcal{F} = \{S_1, S_2, \dots, S_m\}$, tal que: $\mathcal{X} = \bigcup_{i=1}^m S_i$, con $S_i \in \mathcal{F}$; se desea determinar la cantidad mínima de conjuntos de $\mathcal{F}$ cuya unión forme el universo $\mathcal{X}$.

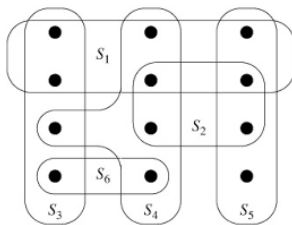
- Definimos como un Set Cover (SC) $\mathcal{C}$ de $\mathcal{X}$, a cualquier subconjunto de $\mathcal{F}$ que contenga a todos los elementos de $\mathcal{X}$.
- Llamamos al SC para $\mathcal{X}$ de tamaño mínimo, como la solución óptima y por tanto una solución para MSCP.
- Considere que MSCP es un problema NP-Hard $\Rightarrow$ a la fecha no se ha encontrado algún algoritmo que en tiempo polinomial encuentre la solución óptima.

Ejemplos MSCP



$$\mathcal{F} = \{A, B, C, D, E\}$$

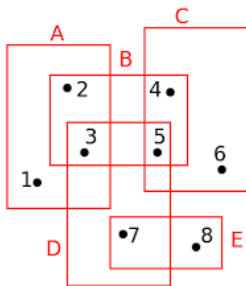
$$\text{MSC} = |\{A, C, E\}| = 3$$



$$\mathcal{F} = \{S_1, S_2, S_3, S_4, S_5, S_6\}$$

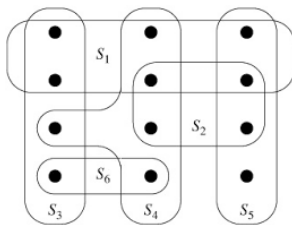
$$\text{MSC} = |\{S_3, S_4, S_5\}| = 3$$

Ejemplos MSCP



$$\mathcal{F} = \{A, B, C, D, E\}$$

$$MSC = |\{A, C, E\}| = 3$$



$$\mathcal{F} = \{S_1, S_2, S_3, S_4, S_5, S_6\}$$

$$MSC = |\{S_3, S_4, S_5\}| = 3$$

Escriba el pseudocódigo de la solución exhaustiva que resuelve MSCP y determine su tiempo de complejidad.

Algoritmo Greedy para MSCP

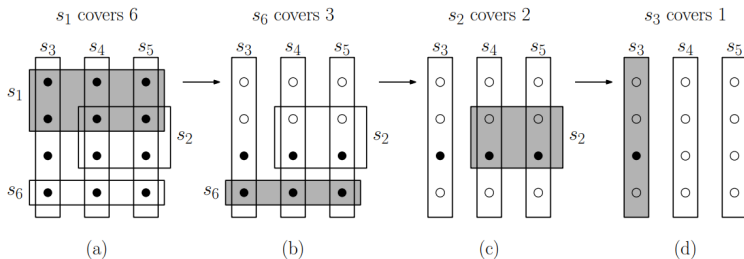
- Lo que se suele hacer con problemas NP-Hard, es diseñar algoritmos que determinen una solución aproximada; es decir, que dan una solución correcta pero no la mejor necesariamente.

Estrategia greedy e iterativa, diseñada por Tomás Cormen [1]¹ para el MSCP:

1. Se comienza con un universo U con todos los elementos: $U = \chi$, e iteramos mientras $U \neq \emptyset$
2. En cada paso, se incluye el subconjunto $S_i \in \mathcal{F}$, que aporta mayor cantidad de elementos a cubrir, entre los disponibles; es decir, aquel que maximiza $|U \cap S_i|$
3. Eliminamos los elementos de S_i del universo que resta por cubrir: $U = U - S_i$ y avanzamos a la siguiente iteración.

¹[1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to Algorithms. The MIT Press, 2nd edition, 2001.

Ejemplo Greedy-Cormen MSCP



Note que el algoritmo óptimo encuentra la solución $\mathcal{C}^* = \{s_3, s_4, s_5\}$ de tamaño 3.

Greedy-Cormen MSCP - Pseudocódigo

Algorithm 1 GREEDY-SETCOVER algorithm in [5] to compute a set cover of the universe $\mathcal{X}$, where $\mathcal{X}$ is the union of all subsets in $\mathcal{F}$

Require: The universe of elements $\mathcal{X}$ and the family of subsets $\mathcal{F}$ whose union is $\mathcal{X}$

```
1: procedure GREEDY-SETCOVER( $\mathcal{X}, \mathcal{F}$ )  
2:    $\mathcal{U} = \mathcal{X}$   
3:    $\mathcal{C} = \emptyset$   
4:   while  $\mathcal{U} \neq \emptyset$  do  
5:     select  $S \in \mathcal{F}$  that maximizes  $|\mathcal{U} \cap S|$   
6:      $\mathcal{U} = \mathcal{U} - S$   
7:      $\mathcal{C} = \mathcal{C} \cup S$   
8:   end while  
9:   return  $\mathcal{C}$   
10: end procedure
```



[5] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to Algorithms, 3rd Edition. MIT Press, 2009.